Exercise Sheet 6 – Fault Tolerance

Lab on 14.1.26 and 16.1.26

In this lab you will implement code related to fault tolerance. In Task 1 you'll implement a version of the message board system that continues to work and does not lose updates (in many cases) when a server crashes and is restarted some time later. Your servers implement this by retrying update operations and by writing checkpoints. Task 2 is about implementing remote procedure calls with at-least-once and at-most-once semantics for the case that a portion of the WebSocket messages are lost during transmission.

You have to submit your solutions by showing them to your tutor. The latest submission is in the lab in the last week of the lecture period (see table below). You can also submit in the lab with the question time on the exercise sheet.

	Presentation of the tasks	Question time	Last possible submission
Group 1:	Wednesday, 14.1.26	Wednesday, 21.1.26	Wednesday, 28.1.26
Group 2:	Friday, 16.1.26	Friday, 23.1.26	Friday, 30.1.26

For the tasks of this exercise sheet you can get 8 points in total. You get it for:

	Task 1a)	Task 1b) + c)	Task 2a) + b)	Task 2c) + d)		
Points:	1	3	2	2		

Task 1 Checkpointing and retrying update operations

Implement a version of the message board system that does not lose update operations and remains consistent (in most cases) when a server crashes and is restarted. Checkpoints are used to not lose the message board content when crashing. Checkpoints are written after each update operation and are reloaded when the server restarts. To not lose updates operations called in another server, the calling server recognizes, that the operation fails, because the connection to the other server could not be established. In this case, the server retries the update operation until it is successful. Note that, with the approach from this task, updates can still get lost when a server crashes. For example, if a server crashes, when it retries to call update operations on another server, then these updates are lost. Ignore this issue.

a) Extend `AsyncBoardStorage.py` with checkpointing.

We implement a simple approach: After each update operation (`put()`, `modify()`, `delete()`, and `deleteAll()`) a checkpoint is written. When the server is started, the last checkpoint is loaded. A checkpoint is written by writing the entire message board (a list of messages) to a JSON file. We simply overwrite the previous checkpoint file. When an object of `AsyncBoardStorage.py` is created, its constructor loads its message board from the JSON file (if it exists). If you do not know how to read and write JSON files, search for examples on the Internet. Include the ID of the server into the filename of the checkpoint (e.g. `checkpoint_1.json` for server 1), as each server needs its own checkpoint file.

Parameters are given to the constructor of class `storage` in `AsyncBoardStorage.py` to inform it about the server ID and whether checkpointing should be used or not. The required signature of the constructor is shown below. Each parameter must have a default value, as older versions of our message board servers call it without parameters. If the parameter `checkpointing` is `True`, checkpoints should be written and read. Parameter `serverID` contains the ID of the server or `None`.

```
def __init__(self, comperatorForElements=None,
                   checkpointing=False,
                   serverID=None):
```

Test the checkpointing with the algorithm `SendToCoordinatorAndBackToServers.py` from Lab 2, Task 3. Make sure, that it still works after all the changes from Labs 3-5. You might need to add the parameter `sequenceNumber=None` to the methods `put()`, `modify()`, `delete()`, and `deleteAll()`, as the stub passes this parameter since Lab 5, Task 1.

To test checkpointing, start 4 servers and use `UserInterface.py` to upload some data. Then close a server and restart it. Connect to it with `UserInterface.py` to inspect the message board. Does the server still contain the same messages? You start a server with `SendToCoordinatorAndBackToServersWithCheckpoint_Main.py` that creates the required objects needed and passes the parameters `serverID` and `checkpointing=True` to the constructor in `AsyncBoardStorage.py`. You get the file in the ZIP archive `DSLabs6_GivenFiles.zip` from StudIP. Download the ZIP archive and put all its files into the directory containing your server files from the previous labs.

Note that this approach for checkpointing is simple but neither reliable nor efficient. As we simply overwrite checkpoint files, files will be corrupted, if the server crashes while the file is written. In real implementations, a new checkpoint file would be written and older files would be deleted later. The approach is inefficient, as we write a checkpoint (the entire message board content) to a file whenever the message board changes. Real implementations would write a checkpoint periodically and record later changes in a log file. Feel free to implement that, if you consider the solution from this subtask as too inefficient and too unreliable.

b) Implement `FaultTolerantProxy.py` that repeats update operations if the destination server is not available.

If a server calls an update operation in another server, then the server must repeat the operation periodically, until it is successful. You will implement that in a new module `FaultTolerantProxy.py`. It contains a class `storage` that has the same interface as `AsyncBoardProxy.py` and can be used as reliable proxy. To (unreliably) call operations in another server, it uses the methods from `AsyncBoardProxy.py`. Hence, as illustrated in Figure 1, `FaultTolerantProxy.py` is as an additional layer between `AsyncBoardProxy.py` and the algorithm for propagating update operations to other servers (`SendToCoordinatorAndBackToServers.py`) for adding reliability.

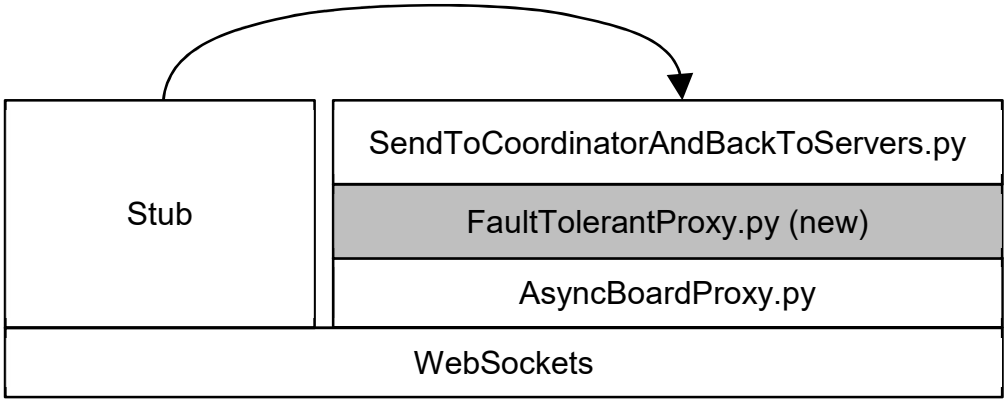

Figure 1: FaultTolerantProxy to call remote procedures in servers by calling methods of AsyncBoardProxy

Start with the file `FaultTolerantProxy.py` that you have downloaded in `DSLabs6_GivenFiles.zip` for subtask a). Its constructor gets an object of `AsyncBoardProxy.py` as parameter which is used to unreliably call operations on a server. As in `UseMutexForUpdates.py`, the update methods (`put()`, `modify()`, `delete()`, and `deleteAll()`) put tuples in a queue (`asyncio.Queue()`). A coroutine that runs concurrently reads the tuples from the queue and calls the corresponding operations via the `AsyncBoardProxy` on the server. If `AsyncBoardProxy` signals that the connection to the server cannot be established, then the operation is called again some time later (e.g. after a second). This is repeated until the operation is successful. The non-update operations (e.g. `get()`, `getNum()`, `getBoard()`, …) simply call the corresponding operation in `AsyncBoardProxy`, as their results are needed immediately.

Start `SendToCoordinatorAndBackToServersFaultTolerantProxy_Main.py` to start a server. You downloaded it in subtask a) in the ZIP file `DSLabs6_GivenFiles.zip`. It creates all objects an connects them. An object of class `FaultTolerantProxy.py` is given to `SendToCoordinatorAndBackToServers.py` as proxy. It is connected to proxies from `AsyncBoardProxy.py` to communicate with another server. Note that the server with ID 0 is the coordinator. All other servers send their update operations to it, before it sends the update operations back to all servers.

c) Test your servers thoroughly with and without server failures.

Do first tests with all four servers running (no failures). Use `UserInterface.py` to do first updates. Can you also see the updates on other servers? Also upload 4 messages to each server using `SendDataToSeveralServers.py`. Do all servers contain all messages in the same order? Take notes about your results of this and in the following tests.

Do further tests with a missing server and `UserInterface.py`. Start all servers except the server with ID 3. Connect to the server with ID 2 (port 10002). Do update operations and observe the behavior of the servers. Use the command getboard, to observe the messages on the server (ID 2). Are these correct? Connect to the server with ID 1 (port 10001). Does the command getboard show the same messages? Then, start the server with ID 3, does it synchronize? Connect to it and check its message board.

Repeat the entire experiment with the 3 servers with ID 1, 2, and 3 are started. Hence, the coordinator is missing. What happens? Is the behavior correct? Restart the coordinator (with ID 0)? Do you then observe all messages when connecting to the servers with ID 1 and 2?

Test what happens if you terminate one of the servers, while many updates happen. Modify the program `SendDataToSeveralServers.py` to send messages only to the servers with ports 10001 and 10002. Therefore, you have to change the initialization of the variable serverPorts to `serverPorts = [10001, 10002]`. Start all four servers. Start `SendDataToSeveralServers.py` to send 1000 messages. After a few seconds, terminate the server with ID 3 (port 10003). After the remaining servers have finished their updates, use UserInterface.py to connect the servers on ports 10001 and 10002. Do these contain the 2000 messages that were uploaded? Restart the server with ID 3. Does it show synchronization activities? Connect to it with UserInterface.py. Does the server also contain 2000 messages? If not, can you explain why?

Repeat the same experiment. But this time you terminate and restart the coordinator (ID 0, port 10000) instead of the server with ID 3. Which messages do you observe on the servers, when the coordinator is terminated? Do you get all 2000 messages, after restarting the coordinator? If not, can you explain why?

Task 2 At-least-once and at-most-once semantics

Implement remote procedure calls with at-least-once semantics and, for those procedures for which it is necessary, with at-most-once semantics. Make sure that you have understood these semantics (lecture slides 09, from slide 33). In contrast to Task 1, this task is not about recovering from server crashes. It is about dealing with messages that have been lost during transmission between two servers. With some network protocols messages can get lost randomly (e.g. with UDP). However, this is not the case for WebSocket connections.

To be able to do experiments with lost messages, we simulate the message loss with the new module `LossyWebsocket.py`. It uses the WebSocket API to send and receive messages as a client. It provides a similar API for sending and receiving messages but 10% of the messages are lost (in both directions). As illustrated in Figure 2, `AsyncBoardProxy.py` can use it after minor modification. In this case 10% of its requests and responses will get lost. In both cases the called remote procedure will never finish, as the proxy is waiting for the response infinitely.

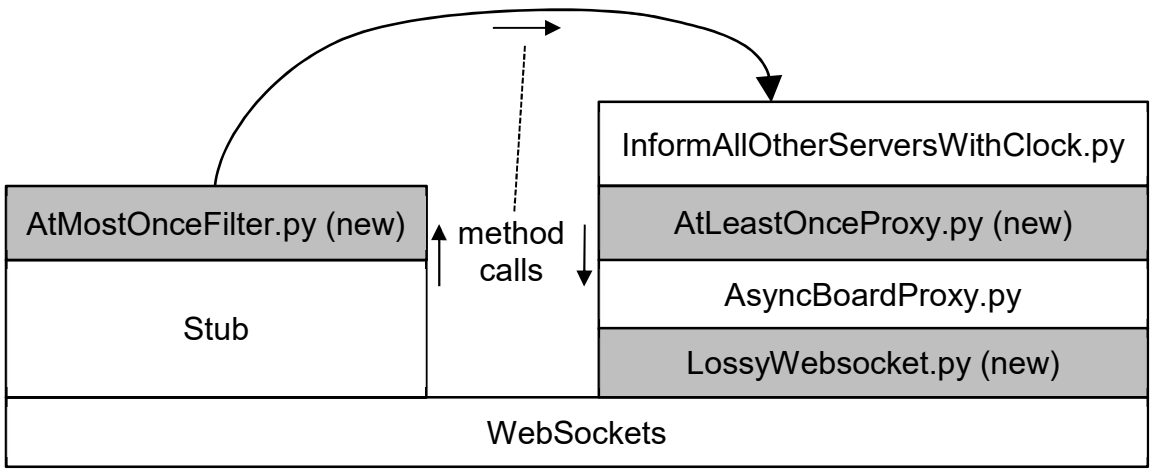

Figure 2: Architecture for at-most-once semantics

Timeouts are used to handle the infinite waiting of a remote procedure. A procedure is given a time limit (e.g. 3 seconds) until which it must have terminated. If it has not

terminated until then, it is aborted and restarted. This makes the proxy send a second request message after which it waits for a response. Again, there is the time limit, after which the process is repeated. This implements the at-least-once semantics, as the remote procedure will eventually be called. We implement that in subtask b) in the module `AtLeastOnceProxy.py` that calls methods in `AsyncBoardProxy.py` to do the actual communication with the server. It has the same interface as `AsyncBoardProxy.py` and can hence be used by an algorithm for distributing messages of our message board system as shown in Figure 2. For this task we use the algorithm in `InformAllOtherServersWithClock.py` with the vector clock.

The issue with the at-least-once semantics is, that remote procedures can be executed on the server more than once. This happens when the response was lost after the procedure has already been executed on the server. As the client cannot know whether the request or the response was lost, it calls the procedure again. The multiple execution is not a problem for procedures that are idempotent, that is, for which is has the same effect, if it is called once or several times. For instance, for our message board system, this is the case for `get()` and `getNum()`. However, for `put()` it makes a difference if it is called one or several times, as a message is then also added to the message board once or several times.

For this reason, non-idempotent remote procedures need the at-most-once semantics. Its implementation includes the mechanism for repeating calls after a timeout from the at-least-once semantics. However, to avoid multiple executions of a procedure, the server recognizes that the procedure was called from the client for a second time. In this case it does not execute the procedure again, but returns the response from a cache. The repeated call can be recognized by numbering each call from a client with a sequence number (1, 2, 3, …). We will implement that filtering of repeated calls in subtask c) in the module `AtMostOnceFilter.py`. It has the same interface as `InformAllOtherServersWithClock.py` and can hence be called from the stub as shown in Figure 2.

Several files needed for this task were given in the ZIP archive `DSLabs6_GivenFiles.zip` that you have downloaded from StudIP and unzipped for Task 1. It is `LossyWebsocket.py` for simulating lost WebSocket messages, as well as the modules `AtLeastOnceProxy.py` and `AtMostOnceFilter.py` with empty classes to be completed in subtasks b) and c). It also contains four `…_Main.py` files for starting your servers in subtasks a), b), c), and d) (`InformAllOtherServersWithClockLossy_Main.py`, `InformAllOtherServersWithClockAtLeastOnce_Main.py`, `InformAllOtherServersWithClockAtMostOnce_Main.py`, and `SendToCoordinatorAndBackToServersAtLeastOnce_Main.py`).

a) Change `AsyncBoardProxy.py` such that it can use `LossyWebsocket.py`.

   Your implementation of `AsyncBoardProxy.py` uses the coroutine function `connect()` to establish a connection to another server via WebSockets. The returned object has the coroutine methods `send()` and `recv()` to send and receive messages. The given module `LossyWebsocket.py` has a similar interface. It also has a function `connect()` that returns an object with the same methods for sending and receiving messages, however losing 10% of it. Hence, we could just replace the call to function `connect()` from the WebSocket library by a call to the one from `LossyWebsocket.py`.

   Make the call to `connect()` configurable in `AsyncBoardProxy.py`, to allow that the module can still be used with other versions of the message board system. To do so, add one more parameter to the constructor of class storage in `AsyncBoardProxy.py`. It

needs to be named `websocketconnect` and must have a default value that is the function `connect()` from the `websockets` library, to support other message board versions. Declare the parameter as `websocketconnect=client.connect`, assuming that you did an import with `from websockets.asyncio import client`. Use the function given as parameter to connect to the server.

For this subtask, execute `InformAllOtherServersWithClockLossy_Main.py` to start a server. It creates the required objects needed for the server. It also passes the function `connect()` from `LossyWebsocket.py` to the constructor in `AsyncBoardProxy.py`. It neither includes `AtLeastOnceProxy.py` nor `AtMostOnceFilter.py` into the architecture.

To test your code, start a cluster of 4 servers. Use `UserInterface.py` to add some messages to the message board with command put. The client will freeze, when a message is lost. Observe the server logs to understand which message was lost. For testing, you may change the probability for losing messages in `LossyWebsocket.py` (Variable `lossProbablity`). If you set it to 0.0, you should be able to use the message board without problems. Set it back to 0.1 for the experiments in the subsequent subtasks. Note that a message loss probability of 0.1=10% is very high. It was chosen to ensure, that you see missing packets quickly.

b) Implement at-least-once semantics for all operations with timeouts and retransmission.

To do so, implement `AtLeastOnceProxy.py`. An empty class just forwarding each method call to `AsyncBoardProxy.py` is given. It is recommended to first only implement and test method `put()`. When this works, implement the other methods (`get()`, `getNum()`, `getBoard()`, `modify()`, `delete()`, and `deleteAll()`). You do not need to implement the methods that were added for mutex, leader election, and sequencer.

For each remote procedure, implement a timeout of 3 seconds. You call a remote procedure via `AsyncBoardProxy.py`. A reference to its object is available in `AtLeastOnceProxy.py` as attribute `self.proxy`. If a function has not finished its execution after 3 seconds, it needs to be aborted. For coroutines, you implement this with `asyncio.timeout()`. For example, a coroutine function `f()` can be given a timeout of 3 seconds with the code shown below. If `f()` has not finished its execution after 3 seconds, it is aborted and an `asyncio.TimeoutError` exception is raised. You need to handle the exception.

```
async with asyncio.timeout(3):
    return await f()
```

Use a loop to repeat the execution of the remote procedure. However, abort after 5 trials. If there would not be a limit, your code would do an infinite loop, if the error is persistent, e.g. if the remote server had crashed.

For testing, start four servers by executing the module `InformAllOtherServersWithClockAtLeastOnce_Main.py`. It adds objects of `LossyWebsocket.py` and `AtLeastOnceProxy.py` to the architecture but does not include `AtMostOnceFilter.py`. Use UserInterface.py to test all 7 operations. Also start `SendDataToSeveralServers.py` to create 4 new messages on each server. How many messages do you observe on each server? Do all servers contain the same messages? Are these in the correct order? Can you explain why, if this is not the case?

c) Implement the at-most-once semantics with duplicate filtering and retransmission of responses for remote procedures that are not idempotent.

Determine which remote procedures are idempotent. Your experiments in subtask b) surely revealed that `put()` is not idempotent, as its repeated calls created messages several times. Which of our other remote procedures (`get()`, `getNum()`, `getBoard()`, `modify()`, `delete()` und `deleteAll()`) are not idempotent? Only implement the at-most-once semantics for these.

Implement the at-most-once semantics in module `AtMostOnceFilter.py`. An empty class just forwarding each method call is given. In `AtLeastOnceProxy.py` you have already implemented to repeat calls if a message got lost. For the at-most-once semantics, `AtMostOnceFilter.py` has to filter duplicated calls and return the response from the first call instead. To be able to do so, it gets the (potentially duplicated) calls from the stub and forwards the calls to `InformAllOtherServersWithClock.py` (see Figure 2). The given attribute `self.proxy` contains a reference to the object from the class in `InformAllOtherServersWithClock.py`.

To be able to identify repeated calls of remote procedures, add sequence numbers (1, 2, 3, …) to non-idempotent procedures in module `AtLeastOnceProxy.py` that you have implemented in subtask b). Pass the sequence number in the parameter with the same name that we have already added in Lab 5, Task 1 to transmit sequence numbers generated by the sequencer.

In `AtMostOnceFilter.py`, implement a cache to remember previous calls and their responses. Identify calls by their sequence number and the ID of the calling server. If the cache already contains the call, do not call the method in `InformAllOtherServersWithClock.py` again. Return the response from the previous call instead. When can responses be deleted from the cache again?

For testing, start four servers by executing the module `InformAllOtherServersWithClockAtMostOnce_Main.py` that creates the architecture shown in Figure 2. Upload some messages with put using `UserInterface.py`. Do you observe cases in which remote procedure calls are repeated? Has the message been created only once anyway? As final test, start `SendDataToSeveralServers.py` to create 4 new messages on each server. Do all four servers contain the same 16 messages in the same order afterwards? Can you observe repeated calls that were filtered?

d) Repeat the tests of your new modules for at-least-once and at-most-once semantics with the update distribution algorithm `SendToCoordinatorAndBackToServers.py`.

Use the given file `SendToCoordinatorAndBackToServersAtMostOnce_Main.py` to start four severs. When running `SendDataToSeveralServers.py` uploading four messages to each server, do all four servers contain the same 16 messages in the same order afterwards? If not, use `UserInterface.py` to inspect which messages are on the servers, as the servers might not have finished processing, when `SendDataToSeveralServers.py` downloaded the messages boards from the servers. If one or several messages are still missing, can you explain why?